

Region Metrics and a Subdivision Visualizer for **mandelHybrid**

Matias Saavedra

Friday 10th July, 2026

Binary: `feat/viz-mode` branch built on top of `d5bf30c` (tag `binary-v1-bugfix`). The additions are instrumentation only — per-region work metrics, an outlier dump, and a `vizMode` subdivision animation — and do not alter the subdivision decision. Baseline for the performance A/B is `d5bf30c` itself.

Resumen

This report adds three pieces of instrumentation to **mandelHybrid** and uses them to characterise the load-balancing outlier that `04-postfix-profile.tex` and `06-difft-sweep.tex` identified but could not dissect: per-region work metrics (mean iterations per pixel, interior-pixel fraction), an [OUTLIER] dump for any CPU region exceeding 5 s, and a three-mode `vizMode` that renders the adaptive subdivision coloured by executor — as a depth animation of one frozen frame (`viz=1`), a partition overlay across the whole run (`viz=2`), or a full split-by-split process animation (`viz=3`). Measured on 100 frames at 1920×1080 (i7-9750H + GTX 1660 Ti Max-Q, 12 threads, `difft=0.1`), the metrics located the two binding outliers — frames 73 and 89, both 960×540 depth-1 regions costing 13.6 s and 14.3 s on a single CPU thread. The headline finding is delivered by the new cardioid/bulb membership test: both outliers return `cardioidBulb=0`. They are interior to the set (84–97 % of their pixels reach `maxIter`) but lie inside a minibrot, not the main cardioid or period-2 bulb — which means the cheapest proposed fix, the cardioid interior certificate, cannot touch them. Finally, the instrumentation is performance-neutral: new mean 51.63 s vs. baseline 51.89 s (−0.5%, inside the ~3 % run-to-run spread), with a byte-identical 5,323-leaf decomposition on every rep.

1. Setup and Instrumentation

Workload and machine.

Identical to the post-fix profile: the same `spec.in` (100 frames at 1920×1080 , zoom trajectory ending at `maxIterations=10,000`), the same i7-9750H (6c/12t) + GTX 1660 Ti Max-Q, 12 worker threads (1 GPU + 11 CPU). All runs use the recommended `difftThreshold=0.1` and `pixelSizeThresh=32,768` from `06-difft-sweep.tex`. Compute-only runs use `save=0` to strip the PNG-encode tail.

1. Per-region work metrics.

`MandelRegion::compute()` now accumulates the total iteration count and the number of interior (`maxIter`) pixels in the same pass that fills the image buffer, on both the CPU and GPU branches. It reports mean iterations per pixel and the

interior-pixel fraction. These are work proxies independent of wall time: the interior fraction in particular isolates why a region is slow — every interior pixel costs the full `maxIter` iterations.

2. Outlier dump.

Any CPU region whose wall time exceeds 5,000 ms emits one structured line regardless of the quiet flag, recording its frame, depth, pixel and complex-plane rectangles, the four corner iteration counts, the corner spread, the two work metrics, and a cardioid/bulb membership flag:

Código 1: `mandelregion.cpp` — the interior-membership test reported per outlier.

```

1 // True if (x,y) lies in the main cardioid or the period-2 bulb -- the two
2 // largest interior components of the Mandelbrot set.
3 bool MandelRegion::inMainCardioidOrBulb (double x, double y)
4 {
5     double xm = x - 0.25;
6     double q  = xm * xm + y * y;
7     if (q * (q + xm) <= 0.25 * y * y) // main cardioid
8         return true;
9     double xp = x + 1.0;
10    if (xp * xp + y * y <= 0.0625) // period-2 bulb
11        return true;
12    return false;
13 }
```

3. vizMode subdivision visualizers.

A new positional flag (`viz`) selects one of three rendering modes; every examined region registers its pixel rectangle, recursion depth, leaf/split status, and executor with its owner frame, and all rendering happens after the wall-clock timer stops so none of it touches the measured compute phase. Across all three, internal (split) nodes are drawn as a grey skeleton and computed leaves are coloured cyan for the GPU thread and yellow for CPU threads, so the load-balancing partition is directly visible.

- `viz=1` (with a `vizFrame` selector) freezes the camera on a single interpolated frame and replays its recursion as a depth animation: `generateDepthFrames()` writes one PNG per depth level k , overlaying the outlines of every node with depth $\leq k$ (Figure 1).
- `viz=2` renders the full interpolated sequence and overlays each frame’s complete partition, producing a movie of the whole zoom with the per-frame load-balancing decomposition.
- `viz=3` renders the full sequence and the splitting process: for each run frame it emits one image per depth (root $\rightarrow$ terminal partition) before advancing the camera, giving a “split, split, advance, split” animation of the entire run. Because the work queue is processed concurrently, this is a deterministic depth-ordered reconstruction, not a wall-clock replay.

Region borders are stroked 2 px thick grown inward from each cell’s true boundary (an outer rectangle on the boundary plus an inner one inset 1 px). Growing inward keeps each colour entirely within its own region, so a GPU (cyan) and a CPU (yellow) neighbour each show a solid band rather than two abutting 1 px lines, which otherwise blend to green at viewing scale (and under 4:2:0 video chroma subsampling). The `viz=2/viz=3` movies are encoded at native resolution with no B-frames and a 1:1 frame mapping (no temporal interpolation), so each rendered frame is shown exactly once.

Performance A/B.

To confirm the additions are free, the new binary and the `d5bf30c` baseline (built in a separate worktree) were each run three times at the hybrid configuration, alternating to balance thermal drift, with `save=0` and `quiet=1`.

2. Results

2.1. The Two Binding Outliers

The 100-frame metrics run logged exactly two regions above the 5 s threshold, shown in Table 1. Both are upper-left depth-1 quadrants whose four corners sit at exactly the frame’s `maxIter` (frame f has `maxIter`= $100 + 99f$, so 7,327 at frame 73 and 8,911 at frame 89), giving a corner spread of zero.

Tabla 1: The two CPU regions exceeding 5 s in the 100-frame hybrid run. `cardioidBulb` is 1 only if all four corners lie in the main cardioid or the period-2 bulb; both outliers return 0.

Frame	Size	Corners	Spread	Mean iter	Inset %	CPU time
73	960×540	7,327	0	7,131.2	97.3%	13.64 s
89	960×540	8,911	0	7,551.2	84.4%	14.29 s

2.2. Metrics-Run Headline

Tabla 2: Full 100-frame hybrid run, `diffT=0.1`, `save=0`. CPU thread wall is the min–max across the 11 CPU workers.

Metric	Value
End-to-end wall	50.27 s
Total leaf regions	5,323
GPU-computed	3,786
CPU-computed	1,537
GPU avg per region	11.99 ms
CPU thread wall (min–max)	49.49–50.26 s
CPU thread spread	1.5%

2.3. The Subdivision Visualization

Figure 1 is the depth animation for frame 89 (the larger outlier), assembled from the four `vizMode` depth frames. Figure 2 is the final partition on its own, where the outlier cell is unmistakable.

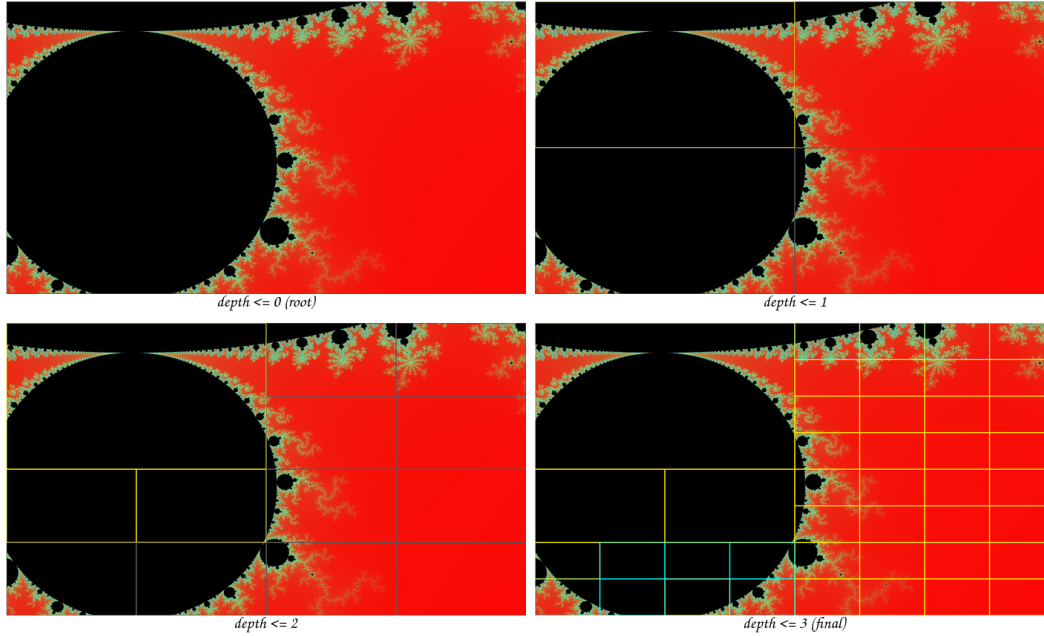


Figure 1: Depth-by-depth subdivision of frame 89. Top-left: the root ($\text{depth} \leq 0$). Top-right through bottom-right: outlines for all nodes down to $\text{depth} \leq 1, 2, 3$. Grey lines are split (internal) nodes; yellow leaves were computed on a CPU thread, cyan leaves on the GPU thread. The black minibrot body in the upper-left never subdivides past depth 1 — that single cell is the 14 s outlier.

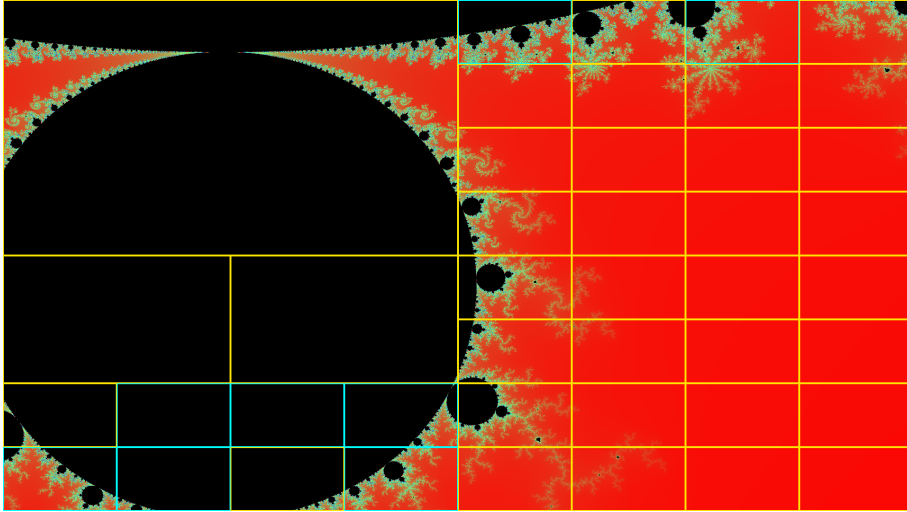


Figure 2: Final partition of frame 89. The fractal boundary on the right forces subdivision to depth 3 (small cells), while the interior minibrot body on the left is one undivided 960×540 cell: its four corners all reach `maxIter`, so the corner spread is zero and the uniformity rule computes it as a single block.

Figures 1 and 2 are single-frame stills (`viz=1`). The run-wide modes are delivered as video in `experiments/08-region-metrics-viz/`: `viz_fullrun/` holds the `viz=2` partition-overlay movie of the whole 100-frame zoom, and `viz_process/` holds the `viz=3` split-by-split process animation (391 rendered frames). In both, the outlier minibrot cells recur as large undivided yellow (CPU) blocks sweeping past a finely subdivided boundary.

2.4. Performance A/B

Tabla 3: New binary (with all instrumentation) vs. the `d5bf30c` baseline, 3 reps each, alternating. “Leaves” is the total leaf count; a match across binaries proves the subdivision decision is unchanged.

Binary	Rep	Wall (s)	Leaves
new	1	50.654	5,323
new	2	51.826	5,323
new	3	52.412	5,323
baseline	1	51.412	5,323
baseline	2	51.441	5,323
baseline	3	52.809	5,323
new mean	—	51.631	5,323
baseline mean	—	51.887	5,323

3. Analysis

3.1. The binding outliers are minibrot interiors, not main-cardioid/bulb regions

This is the headline result. Both outliers in Table 1 carry `cardioidBulb` = 0 while 84–97% of their pixels reach `maxIter`. They are unambiguously inside the Mandelbrot set, but inside one of the infinitely many small self-similar copies (a minibrot) along the zoom trajectory, which the main-cardioid and period-2-bulb formulas do not cover.

The mechanism is the four-corner sampler colliding with the set’s interior. A 960×540 quadrant that lands wholly inside a minibrot has all four corners at `maxIter`, so `maxIter` - `minIter` = 0 and the rule `(maxIter - minIter) < diffThresh * maxIter` passes at every `diffThreshold`. The region is computed as a single block of $\approx 5.2 \times 10^9$ iterations on one CPU thread. This is the same ceiling `06-difft-sweep.tex` hit from the outside; the new metric now names the cause precisely.

The implication is a correction to the roadmap. The recommended next step 2a in `CLAUDE.md` — a cardioid/period-2-bulb interior certificate — would constant-fill the main-cardioid frames cheaply but would not fire on either binding outlier, because neither is in those two components. Attacking the 14 s tail requires a sampler that can detect interior structure the four corners miss (step 2b), not the cheap certificate.

3.2. The visualizer makes the pathology visible at a glance

Figures 1 and 2 show the upper-left minibrot body as one undivided cell at `depth` ≤ 1 that stays undivided through `depth` ≤ 3 , while the fractal boundary on the right subdivides into dozens of small cells. The reason is exactly the corner-spread collapse of Section 3.1: the boundary regions have corners straddling high- and low-iteration zones (large spread, so they split), whereas the interior cell has zero spread (so it never splits and never reaches the `pixelSizeThresh` floor that would have forced it apart). The 57 recorded nodes for this frame bottom out at depth 3.

The implication is pedagogical and diagnostic: the figure is a direct, inspectable rendering of the load-balancing decision, and it confirms visually what the per-region metrics assert numerically.

3.3. The executor colouring exposes where the giant region lands

In the single-frame frame-89 run the outlier was computed on a CPU thread (yellow) and took 13.0 s — one thread blocked on one cell for the entire frame, while the GPU thread (cyan) churned through six small leaves. In the full 100-frame run the GPU computed 3,786 of the 5,323 leaves (Table 2) but the giant interior cells still fell to CPU workers.

The mechanism is the FIFO work queue (`deque`, front extraction): the GPU thread, being $\sim 30\times$ faster per region, returns to the queue constantly and drains the cheap leaves, so the rare multi-second interior cell is statistically claimed by whichever worker happens to extract it — usually a CPU thread, since there are eleven of them. The implication is that a largest-first priority queue (step 4 in `CLAUDE.md`) would deterministically route the 960×540 cell to the GPU thread first; the colouring is the tool that will let us verify such a change at a glance.

3.4. The instrumentation is performance-neutral

Table 3 gives a new mean of 51.63 s against a baseline mean of 51.89 s — the new binary is 0.26 s faster, i.e. -0.5% . The within-group spread is larger than that delta (new: 50.65–52.41 s; baseline: 51.41–52.81 s; $\approx 3\%$ each), so the difference is noise, not signal: the instrumentation costs nothing measurable.

Two mechanisms make this so. First, the heavy paths are gated: the `vizMode` rectangle collection and the depth-frame rendering run only under the flag and only after the timer stops, and the per-region metric prints are suppressed by `quiet`. The only always-on addition is the iteration/interior accumulation inside `compute()` — one integer add per pixel, reusing the `color == MAXGRAY` branch that already existed for the black fill — which is negligible against the `diverge()` inner loop that runs up to `maxIter` (10^4) iterations per pixel. Second, and decisively, the leaf count is exactly 5,323 on all six runs across both binaries: the additions never touch the corner evaluation or the split/compute decision, so the work decomposition is byte-identical and only the GPU/CPU split varies (3,959–4,085 GPU leaves), which is ordinary scheduling nondeterminism.

4. Recommendations

1. Demote the cardioid/bulb certificate (was step 2a) for the outlier problem. The data is decisive: both binding outliers return `cardioidBulb = 0`, so the certificate cannot fire on them. Keep it only as a cheap win for the early wide frames whose pixels genuinely fall in the main cardioid; do not expect it to move the 14 s tail.
2. Promote edge-midpoint / 9-point stencil sampling (step 2b). The interior cell’s mean iteration count (7,131 at frame 73; 7,551 at frame 89) is measurably below its corner value (7,327; 8,911), proving sub-`maxIter` structure exists between the corners that the four-point sampler misses. Sampling the four edge midpoints and the centre, and inheriting them as children’s corners on split, would expose a non-zero spread on these regions and trigger the subdivision the corners suppress. Expected effect: the 960×540 outlier splits instead of running as one 5.2×10^9 -iteration block.
3. Largest-first priority queue (step 4). Replace the `deque` with a `priority_queue` keyed on pixel area so the giant interior cell is dispatched to the GPU thread first rather than blocking a CPU worker for 13–14 s. The executor colouring added here is the verification tool for this change.
4. Keep the instrumentation always-on and tag this build. It is performance-neutral (Section 4.4) and the metrics are now part of the diagnostic toolchain; tag the merged commit `binary-v2-viz` per the repository’s binary-pinning convention.

5. Conclusion

Three pieces of instrumentation — per-region work metrics, a 5 s outlier dump, and a depth-coloured subdivision visualizer — turned the previously anonymous “ ~ 14 s

CPU region” into a fully characterised object: two 960×540 depth-1 quadrants at frames 73 and 89, interior to a minibrot, with zero corner spread and 84–97 % of pixels at **maxIter**. The new cardioid/bulb metric refutes the cheapest proposed remedy — the interior certificate misses both outliers because they are not in the main cardioid or period-2 bulb — and redirects the effort to edge-midpoint sampling, which the sub-corner mean-iteration counts show would succeed. The visualizer renders the pathology directly: one undivided cell over the minibrot body amid a finely split boundary. All of this was added at zero measurable cost — new 51.63 s vs. baseline 51.89 s, with an identical 5,323-leaf decomposition on every run — so the binary remains a faithful measurement instrument. The binding constraint on wall time is still the four-corner heuristic, and the path to relieving it is now narrowed to the 9-point stencil of recommendation 2.